

Lab 7 Report: AVL Tree [cite: 6, 7]

April 8, 2026

1 Introduction to AVL Tree

In the previous lab, we learned how to build up a Binary Search Tree (BST)[cite: 12]. In this lab, we extend the BST to a Balanced Binary Search Tree known as the AVL Tree[cite: 13]. Named after its authors Adelson-Velskii and Landis (1962), an AVL tree guarantees that for every node, the height of the left and right subtrees can differ only by 1[cite: 25, 26]. An empty tree has a height of -1 [cite: 26].

This difference in height is called the Balance Factor[cite: 27]. A Balance Factor is defined by the height difference of the left subtree and the right subtree[cite: 74, 75]. When the absolute value of the Balance Factor is strictly greater than 1, the tree becomes unbalanced and must be rebalanced[cite: 27].

2 Node Implementation

To implement the AVL tree, we add a `height` attribute to the standard BST `Node` class[cite: 46, 47]. This allows us to access the height of any node in $O(1)$ time[cite: 47].

```
1 public class Node {  
2     Integer key; [cite: 49]  
3     Node left, right; [cite: 50]  
4     int height; [cite: 52]  
5  
6     public Node(Integer key) { [cite: 56]  
7         this.key = key; [cite: 57]  
8         this.height = 0; [cite: 58]  
9         this.left = this.right = null; [cite: 60]  
10    }  
11 }
```

Listing 1: Node Class Implementation

The height and balance factor can be calculated as follows:

```
1 public int height(Node node) { [cite: 65]  
2     if (node == null) return -1; [cite: 70, 71]  
3     return node.height; [cite: 72]  
4 }  
5  
6 private int checkBalance(Node x) { [cite: 76]
```

```

7     return height(x.left) - height(x.right); [cite: 78, 79]
8 }

```

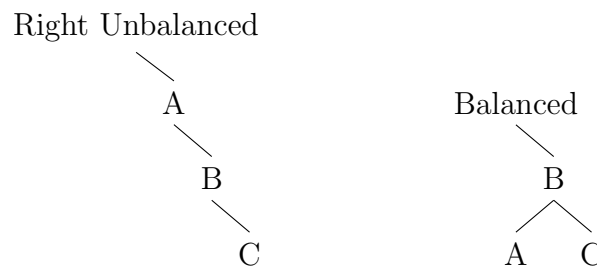
Listing 2: Helper Methods

3 Tree Rotations

Insert or delete operations may cause the AVL tree to become imbalanced[cite: 81]. A simple modification called rotation restores the AVL property[cite: 82]. There are four types of rotations corresponding to four violation cases[cite: 88].

3.1 Left Rotation

When a new node is inserted and creates a right-right-unbalanced-tree, a left rotation is performed[cite: 90, 91].



```

1 private Node rotateLeft(Node x) { [cite: 114]
2     Node y = x.right; [cite: 118]
3     x.right = y.left; [cite: 119]
4     y.left = x; [cite: 120]
5     x.height = 1 + Math.max(height(x.left), height(x.right)); [cite:
121]
6     y.height = 1 + Math.max(height(y.left), height(y.right)); [cite:
121]
7     return y; [cite: 121]
8 }

```

Listing 3: Left Rotation Code

3.2 Right Rotation

When a new node is inserted and creates a left-left-unbalanced-tree, a right rotation is performed[cite: 123, 124].

```

1 private Node rotateRight(Node x) {
2     Node y = x.left;
3     x.left = y.right;
4     y.right = x;
5     x.height = 1 + Math.max(height(x.left), height(x.right));
6     y.height = 1 + Math.max(height(y.left), height(y.right));
7     return y;
8 }

```

Listing 4: Right Rotation Code

3.3 Double Rotations

- **Left-Right Rotation:** Used when a left-right-unbalanced-tree occurs[cite: 153]. First, perform a left rotation on the left child, then a right rotation on the root[cite: 161, 173].
- **Right-Left Rotation:** Used when a right-left-unbalanced-tree occurs[cite: 175]. First, perform a right rotation on the right child, then a left rotation on the root[cite: 188, 201].

4 Rebalancing Logic

The balancing method checks the balance factor and applies the appropriate single or double rotation[cite: 204].

```
1 private Node balance(Node x) { [cite: 205]
2     if (checkBalance(x) < -1) { [cite: 206]
3         if (checkBalance(x.right) > 0) { [cite: 207]
4             x.right = rotateRight(x.right); [cite: 208]
5         }
6         x = rotateLeft(x); [cite: 211]
7     } else if (checkBalance(x) > 1) { [cite: 219]
8         if (checkBalance(x.left) < 0) { [cite: 220]
9             x.left = rotateLeft(x.left); [cite: 220]
10        }
11        x = rotateRight(x); [cite: 223]
12    }
13    return x; [cite: 226]
14 }
```

Listing 5: Balance Method

5 Exercise 1: AVL Tree Insertion

Combining the BST insertion logic from the previous lab with the new balancing mechanism[cite: 235], the complete `insert` method is implemented as follows:

```
1 public Node insert(Node node, Integer key) {
2     if (node == null) return new Node(key);
3
4     if (key < node.key) {
5         node.left = insert(node.left, key);
6     } else if (key > node.key) {
7         node.right = insert(node.right, key);
8     } else {
9         return node; // Duplicate keys not allowed
10    }
11
12    node.height = 1 + Math.max(height(node.left), height(node.right));
13    return balance(node);
14 }
```

Listing 6: AVL Insertion

6 Conclusion

By adding a `height` attribute and utilizing the `balance` logic during insertions and deletions, we successfully extend a standard Binary Search Tree into an AVL Tree, guaranteeing $O(\log n)$ operations.